

Slide 4 - AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.INTERNET" />

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@drawable/shoplist"
        android:label="@string/app_name"
        android:roundIcon="@drawable/shoplist"
        android:supportsRtl="true"
        android:theme="@style/Theme.AppGerencia">

        <activity
            android:name=".SplashActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <activity
            android:name=".MainActivity"
            android:exported="false" />

    </application>

</manifest>
```

O AndroidManifest.xml é responsável por configurar o aplicativo. Ele define permissões, como acesso à internet, o ícone, o nome e o tema do app. Também registra as Activities do sistema. No nosso caso, a SplashActivity é definida como tela inicial através do intent-filter, enquanto a MainActivity é a tela principal que será aberta após a splash.

slide 5 - SplashActivity.java

```
package com.example.appgerencia;

import android.content.Intent;
import android.os.Bundle;
import android.os.Handler;

import androidx.appcompat.app.AppCompatActivity;

public class SplashActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_splash);

        // Tela de carregamento rápida (1.5 segundos)
        new Handler().postDelayed(() -> {
            startActivity(new Intent(SplashActivity.this, MainActivity.class));
            finish();
        }, 1500);
    }
}
```

Explicação dos imports

`import android.content.Intent;`

👉 Usado para trocar de tela no aplicativo.
Nesse código, ele abre a `MainActivity`.

`import android.os.Bundle;`

👉 Armazena informações do estado da Activity quando ela é criada.

`import android.os.Handler;`

👉 Cria um atraso de tempo.
Aqui ele espera 1,5 segundos antes de abrir a próxima tela.

```
import androidx.appcompat.app.AppCompatActivity;
```

👉 Classe base para criar telas Android modernas com suporte da biblioteca AndroidX.

O que a classe faz:

- Herda de `AppCompatActivity`, permitindo criar uma tela Android:

```
public class SplashActivity extends AppCompatActivity
```

- O método `onCreate()` é executado quando a tela abre:

```
protected void onCreate(Bundle savedInstanceState)
```

- Define o layout da splash screen:

```
setContentView(R.layout.activity_splash);
```

- O `Handler()` cria um atraso de 1,5 segundos:

```
new Handler().postDelayed(() -> {
```

- Depois desse tempo, abre a `MainActivity`:

```
startActivity(new Intent(SplashActivity.this, MainActivity.class));
```

- `finish()` fecha a `SplashActivity` para impedir voltar nela:

```
finish();
```

Slide 6 - ShoppingItem.java

```
package com.example.appgerencia;

import java.io.Serializable;

public class ShoppingItem implements Serializable {
    private String name;
    private double quantity;
    private String unit;
    private double price;
    private String location;

    public ShoppingItem(String name, double quantity, String unit, double price, String location) {
        this.name = name;
        this.quantity = quantity;
        this.unit = unit;
        this.price = price;
        this.location = location;
    }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public double getQuantity() { return quantity; }
    public void setQuantity(double quantity) { this.quantity = quantity; }

    public String getUnit() { return unit; }
    public void setUnit(String unit) { this.unit = unit; }

    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }

    public String getLocation() { return location; }
    public void setLocation(String location) { this.location = location; }
}
```

Package

```
package com.example.appgerencia;
```

👉 Define o pacote onde a classe está localizada.

Import

```
import java.io.Serializable;
```

👉 Permite transformar o objeto em um formato que pode ser salvo ou transferido.

No projeto, isso ajuda no armazenamento dos dados.

Slide 7 - dialog_shopping_item.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="20dp">

    <EditText
        android:id="@+id/editTextName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Nome do Produto"
        android:inputType="textCapSentences" />

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
        android:orientation="horizontal">

        <EditText
            android:id="@+id/editTextQuantity"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:hint="Qtd"
            android:inputType="numberDecimal" />

        <Spinner
            android:id="@+id/spinnerUnit"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1.5"
            android:layout_marginStart="8dp" />

    </LinearLayout>

    <EditText
        android:id="@+id/editTextPrice"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
        android:hint="Preço (R$)"
        android:inputType="numberDecimal" />

    <EditText
        android:id="@+id/editTextLocation"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
        android:hint="Local de Compra"
        android:inputType="textCapWords" />

</LinearLayout>
```

```
<EditText
    android:id="@+id/editTextPrice"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:hint="Preço (R$)"
    android:inputType="numberDecimal" />

<EditText
    android:id="@+id/editTextLocation"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:hint="Local de Compra"
    android:inputType="textCapWords" />

</LinearLayout>
```

Layout principal

```
<LinearLayout  
    android:orientation="vertical"
```

👉 Organiza os elementos verticalmente, um abaixo do outro.

Campo Nome do Produto

```
<EditText  
    android:id="@+id/editTextName"
```

👉 Campo onde o usuário digita o nome do produto.

Layout horizontal

```
<LinearLayout  
    android:orientation="horizontal">
```

👉 Cria uma linha horizontal para colocar:

- quantidade
- unidade

lado a lado.

Campo Quantidade

```
<EditText  
    android:id="@+id/editTextQuantity"
```

👉 Campo para informar a quantidade do produto.

```
    android:inputType="numberDecimal"
```

👉 Permite números decimais.

Spinner de Unidade

```
<Spinner  
    android:id="@+id/spinnerUnit"
```

👉 Lista suspensa para escolher a unidade:

- kilos
 - pacotes
 - unidades
 - etc.
-

Campo Preço

```
<EditText  
    android:id="@+id/editTextPrice"
```

👉 Campo para informar o preço do produto.

Campo Local de Compra

```
<EditText  
    android:id="@+id/editTextLocation"
```

👉 Campo onde o usuário informa o local da compra.

Slide 8 - item_shopping.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.cardview.widget.CardView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="8dp"
    app:cardCornerRadius="12dp"
    app:cardElevation="6dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:padding="16dp"
        android:background="@color/itemCardBG"
        android:gravity="center_vertical">

        <LinearLayout
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:orientation="vertical">

            <TextView
                android:id="@+id/textViewName"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:textSize="18sp"
                android:textStyle="bold"
                android:textColor="@color/primaryGreen"
                android:text="Nome do Produto" />

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="horizontal"
                android:layout_marginTop="4dp">
```



```

        <TextView
            android:id="@+id/textViewQuantity"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textSize="14sp"
            android:textColor="#5D4037"
            android:text="2 pacotes" />

        <TextView
            android:id="@+id/textViewPrice"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginStart="16dp"
            android:textSize="14sp"
            android:textColor="@color/accentRed"
            android:textStyle="bold"
            android:text="R$ 0,00" />
    </LinearLayout>

    <TextView
        android:id="@+id/textViewLocation"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="2dp"
        android:textSize="12sp"
        android:textColor="#757575"
        android:text="Local: Supermercado" />
    </LinearLayout>

    <ImageButton
        android:id="@+id/buttonEdit"
        android:layout_width="40dp"
        android:layout_height="40dp"
        android:background="@attr/selectableItemBackgroundBorderless"
        android:contentDescription="Editar"
        android:src="@android:drawable/ic_menu_edit"
        app:tint="@color/primaryGreen" />

    <ImageButton
        android:id="@+id/buttonDelete"
        android:layout_width="40dp"
        android:layout_height="40dp"
        android:background="@attr/selectableItemBackgroundBorderless"
        android:contentDescription="Excluir"
        android:src="@android:drawable/ic_menu_delete"
        app:tint="@color/accentRed" />

    </LinearLayout>
</androidx.cardview.widget.CardView>

```

Cada item do **RecyclerView** usa esse modelo visual.

Explicação resumida

CardView

<androidx.cardview.widget.CardView

👉 Cria um cartão com:

- bordas arredondadas
- sombra (elevação)

app:cardCornerRadius="12dp"

```
app:cardElevation="6dp"
```

LinearLayout principal

```
<LinearLayout
```

```
    android:orientation="horizontal"
```

👉 Organiza os elementos horizontalmente:

- informações do produto
 - botões
-

Nome do produto

```
<TextView
```

```
    android:id="@+id/textViewName"
```

👉 Exibe o nome do produto.

Exemplo:

- Arroz
 - Coca Cola
-

Quantidade

```
<TextView
```

```
    android:id="@+id/textViewQuantity"
```

👉 Mostra quantidade e unidade.

Exemplo:

- 2 pacotes
 - 5 kilos
-

Preço

`<TextView`

`android:id="@+id/textViewPrice"`

👉 Mostra o valor do produto.

Exemplo:

- R\$ 10,00
-

Local da compra

`<TextView`

`android:id="@+id/textViewLocation"`

👉 Exibe onde o produto foi comprado.

Exemplo:

- Local: Mercado
-

Botão Editar

`<ImageButton`

`android:id="@+id/buttonEdit"`

👉 Permite editar o produto da lista.

Botão Excluir

`<ImageButton`

`android:id="@+id/buttonDelete"`

👉 Remove o produto da lista.

Slide 9 - ShoppingAdapter.java

```
package com.example.appgerencia;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ImageButton;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import java.util.List;
import java.util.Locale;

public class ShoppingAdapter extends RecyclerView.Adapter<ShoppingAdapter.ShoppingViewHolder> {

    private List<ShoppingItem> itemList;
    private OnItemClickListener listener;

    public interface OnItemClickListener {
        void onItemClick(int position);
        void onDeleteClick(int position);
    }

    public ShoppingAdapter(List<ShoppingItem> itemList, OnItemClickListener listener) {
        this.itemList = itemList;
        this.listener = listener;
    }

    @NonNull
    @Override
    public ShoppingViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
        View view = LayoutInflater.from(parent.getContext()).inflate(R.layout.item_shopping, parent, false);
        return new ShoppingViewHolder(view);
    }
}
```

```

@Override
public void onBindViewHolder(@NonNull ShoppingViewHolder holder, int position) {
    ShoppingItem item = itemList.get(position);
    holder.textViewName.setText(item.getName());

    String qtyText = String.format(Locale.getDefault(), "%.2f %s", item.getQuantity(), item.getUnit());
    holder.textViewQuantity.setText(qtyText);

    holder.textViewPrice.setText(String.format(Locale.getDefault(), "R$ %.2f", item.getPrice()));
    holder.textViewLocation.setText("Local: " + item.getLocation());

    holder.buttonEdit.setOnClickListener(v -> listener.onItemClick(position));
    holder.buttonDelete.setOnClickListener(v -> listener.onDeleteClick(position));
    holder.itemView.setOnClickListener(v -> listener.onItemClick(position));
}

@Override
public int getItemCount() {
    return itemList.size();
}

public static class ShoppingViewHolder extends RecyclerView.ViewHolder {
    TextView textViewName, textViewQuantity, textViewPrice, textViewLocation;
    ImageButton buttonEdit, buttonDelete;

    public ShoppingViewHolder(@NonNull View itemView) {
        super(itemView);
        textViewName = itemView.findViewById(R.id.textViewName);
        textViewQuantity = itemView.findViewById(R.id.textViewQuantity);
        textViewPrice = itemView.findViewById(R.id.textViewPrice);
        textViewLocation = itemView.findViewById(R.id.textViewLocation);
        buttonEdit = itemView.findViewById(R.id.buttonEdit);
        buttonDelete = itemView.findViewById(R.id.buttonDelete);
    }
}

```

Imports

```
import androidx.recyclerview.widget.RecyclerView;
```

👉 Permite usar o RecyclerView.

```
import android.view.LayoutInflater;
```

👉 Converte o XML em um componente visual.

```
import android.widget.TextView;
import android.widget.ImageButton;
```

👉 Permite controlar textos e botões.

```
import java.util.List;
```

👉 Permite usar listas de produtos.

Classe principal

```
public class ShoppingAdapter extends  
RecyclerView.Adapter<ShoppingAdapter.ShoppingViewHolder>
```

👉 Cria o Adapter do RecyclerView.

O RecyclerView é um componente essencial no Android Studio usado para exibir grandes conjuntos de dados em listas ou grades

Lista de produtos

```
private List<ShoppingItem> itemList;
```

👉 Armazena todos os produtos cadastrados.

Interface de clique

```
public interface OnItemClickListener
```

👉 Define ações de:

- editar
 - excluir
-

Construtor

```
public ShoppingAdapter(List<ShoppingItem> itemList, OnItemClickListener listener)
```

👉 Recebe:

- a lista de produtos
 - os eventos de clique
-

```
onCreateViewHolder()
```

```
View view = LayoutInflater.from(parent.getContext())  
.inflate(R.layout.item_shopping, parent, false);
```

👉 Pega o XML:

item_shopping.xml

e transforma em um card visual.

```
onBindViewHolder()
```

Essa é a parte MAIS IMPORTANTE.

Ela coloca os dados do produto no card.

Pega o produto da posição atual

```
ShoppingItem item = itemList.get(position);
```

Nome

```
holder.textViewName.setText(item.getName());
```

👉 Coloca o nome do produto no TextView.

Quantidade + unidade

```
String qtyText = String.format(Locale.getDefault(),  
"%0.2f %s",  
item.getQuantity(),  
item.getUnit());
```

👉 Junta:

- quantidade
- unidade

Exemplo:

- 2 pacotes
-

Preço

```
holder.textViewPrice.setText(  
    String.format(Locale.getDefault(),  
        "R$ %.2f",  
        item.getPrice());
```

👉 Formata o preço.

Exemplo:

- R\$ 10,50
-

Local

```
holder.textViewLocation.setText(  
    "Local: " + item.getLocation());
```

👉 Mostra o local da compra.

Botões

Editar

```
holder.buttonEdit.setOnClickListener(...)
```

👉 Chama edição do item.

Excluir

```
holder.buttonDelete.setOnClickListener(...)
```

👉 Remove o item da lista.

```
getItemCount()
```

```
return itemList.size();
```

👉 Retorna quantos produtos existem.

ViewHolder

```
public static class ShoppingViewHolder
```

👉 Guarda referências dos componentes do XML.

```
findViewById()
```

```
textViewName = itemView.findViewById(R.id.textViewName);
```

👉 Liga:

- Java
 com
- XML

Slide 10 - MainActivity.java

```
package com.example.appgerencia;

import android.content.SharedPreferences;
import android.os.Bundle;
import android.text.Editable;
import android.text.TextWatcher;
import android.view.LayoutInflater;
import android.view.View;
import android.widget.AdapterView;
import android.widget.AdapterView.OnItemClickListener;
import android.widget.EditText;
import android.widget.Spinner;
import android.widget.TextView;
import android.widget.Toast;

import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.google.android.material.floatingactionbutton.FloatingActionButton;
import com.google.gson.Gson;
import com.google.gson.reflect.TypeToken;

import java.lang.reflect.Type;
import java.text.NumberFormat;
import java.util.ArrayList;
import java.util.List;
import java.util.Locale;

import coil.Coil;
import coil.request.CachePolicy;
import coil.request.ImageRequest;
```

1. Armazenamento de dados

import android.content.SharedPreferences;

👉 Permite salvar dados localmente no celular.

No projeto:

- salva a lista de compras
- mantém os produtos mesmo após fechar o app

2. Controle da Activity

```
import android.os.Bundle;  
import androidx.appcompat.app.AppCompatActivity;
```

👉 Gerenciam a tela principal (**MainActivity**).

- **Bundle** → guarda estado da tela
 - **AppCompatActivity** → classe base das telas Android
-

3. Monitoramento de texto

```
import android.textEditable;  
import android.text.TextWatcher;
```

👉 Detectam alterações em campos de texto.

Exemplo:

- atualizar total quando o usuário digita algo
-

4. Componentes visuais

```
import android.view.LayoutInflater;  
import android.view.View;  
import android.widget.EditText;  
import android.widget.Spinner;  
import android.widget.TextView;  
import android.widget.Toast;
```

Funções:

- **LayoutInflater** → transforma XML em componente visual
 - **View** → elemento visual base
 - **EditText** → entrada de texto
 - **Spinner** → lista suspensa
 - **TextView** → exibe texto
 - **Toast** → mostra mensagens rápidas
-

5. Janela de diálogo

```
import androidx.appcompat.app.AlertDialog;
```

👉 Cria caixas de diálogo.

No projeto:

- adicionar produto
 - editar produto
 - confirmar exclusão
-

6. Lista de produtos

```
import androidx.recyclerview.widget.RecyclerView;  
import androidx.recyclerview.widget.LinearLayoutManager;
```

👉 Controlam a lista dinâmica.

- `RecyclerView` → exibe produtos
 - `LinearLayoutManager` → organiza em coluna vertical
-

7. Botão flutuante

```
import com.google.android.material.floatingactionbutton.FloatingActionButton;
```

👉 Cria o botão "+" para adicionar produtos.

8. Conversão para JSON

```
import com.google.gson.Gson;  
import com.google.gson.reflect.TypeToken;
```

👉 Convertem objetos para JSON.

No projeto:

- transformam `ShoppingItem` em texto
 - salvam no `SharedPreferences`
-

9. Manipulação de listas

```
import java.util.ArrayList;  
import java.util.List;
```

👉 Criam e armazenam listas de produtos.

10. Formatação

```
import java.text.NumberFormat;  
import java.util.Locale;
```

👉 Formata valores.

Exemplo:

R\$ 15,50

11. Biblioteca Coil

```
import coil.Coil;  
import coil.request.CachePolicy;  
import coil.request.ImageRequest;
```

👉 Biblioteca para carregar imagens.

Funções:

- baixar imagens
- armazenar em cache
- evitar carregamentos repetidos

```

public class MainActivity extends AppCompatActivity implements ShoppingAdapter.OnItemClickListener {

    private List<ShoppingItem> shoppingList;
    private ShoppingAdapter adapter;
    private RecyclerView recyclerView;
    private FloatingActionButton fabAdd;
    private TextView textViewTotalItems, textViewTotalPrice;
    private final String PREFS_NAME = "ShopListPrefs";
    private final String LIST_KEY = "shopping_list";
    private final Locale ptBr = new Locale("pt", "BR");

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        loadData();
        preloadImages();

        recyclerView = findViewById(R.id.recyclerView);
        textViewTotalItems = findViewById(R.id.textViewTotalItems);
        textViewTotalPrice = findViewById(R.id.textViewTotalPrice);

        recyclerView.setLayoutManager(new LinearLayoutManager(this));
        adapter = new ShoppingAdapter(shoppingList, this);
        recyclerView.setAdapter(adapter);

        fabAdd = findViewById(R.id.fabAdd);
        fabAdd.setOnClickListener(v -> showItemDialog(null, -1));

        updateTotals();
    }
}

```

Classe principal

public class MainActivity extends AppCompatActivity
implements ShoppingAdapter.OnItemClickListener

👉 Cria a tela principal do aplicativo.

👉 implements ShoppingAdapter.OnItemClickListener

faz a MainActivity receber os eventos do Adapter:

- editar item
- excluir item

Variáveis

private List<ShoppingItem> shoppingList;

👉 Lista que armazena os produtos cadastrados.

private ShoppingAdapter adapter;

👉 Faz a ligação entre os dados e o [RecyclerView](#).

```
private RecyclerView recyclerView;
```

👉 Exibe a lista de produtos na tela.

```
private FloatingActionButton fabAdd;
```

👉 Botão flutuante para adicionar produtos.

```
private TextView textViewTotalItems, textViewTotalPrice;
```

👉 Exibem:

- total de itens
 - valor total da compra
-

```
private final String PREFS_NAME = "ShopListPrefs";
```

```
private final String LIST_KEY = "shopping_list";
```

👉 Chaves usadas para salvar e recuperar dados do [SharedPreferences](#).

```
private final Locale ptBr = new Locale("pt", "BR");
```

👉 Define o formato brasileiro.

Exemplo:

R\$ 15,50

Método [onCreate\(\)](#)

```
protected void onCreate(Bundle savedInstanceState)
```

👉 É executado quando a tela abre.

Define o layout da tela

```
setContentView(R.layout.activity_main);
```

👉 Liga a MainActivity ao XML:

activity_main.xml

Carrega dados salvos

```
loadData();
```

👉 Recupera a lista de produtos salva anteriormente.

Pré-carrega imagens

```
preloadImages();
```

👉 Carrega imagens antes para melhorar desempenho.

Conecta XML ao Java

```
recyclerView = findViewById(R.id.recyclerView);
```

```
textViewTotalItems = findViewById(R.id.textViewTotalItems);
```

```
textViewTotalPrice = findViewById(R.id.textViewTotalPrice);
```

👉 Faz o Java acessar os componentes do XML.

Configura o RecyclerView

```
recyclerView.setLayoutManager(  
    new LinearLayoutManager(this));
```

👉 Organiza os itens verticalmente.

Cria o Adapter

```
adapter = new ShoppingAdapter(  
    shoppingList,  
    this);
```

👉 Liga:

- lista de produtos
 - eventos de clique
-

Define o Adapter

```
recyclerView.setAdapter(adapter);
```

👉 Exibe os produtos na tela.

Configura botão adicionar

```
fabAdd.setOnClickListener(  
    v -> showItemDialog(null, -1));
```

👉 Quando clicar:

- abre a tela de cadastro
-

Atualiza totais

```
updateTotals();
```

👉 Atualiza:

- quantidade de itens
- valor total

Slide 11 - MainActivity

```
private void preloadImages() {
    if (shoppingList == null) return;
    for (ShoppingItem item : shoppingList) {
        String cleanName = item.getName().toLowerCase().trim().replace(" ", ",");
        int lockId = Math.abs(item.getName().toLowerCase().trim().hashCode() % 1000);
        String imageUrl = "https://loremflickr.com/300/300/" + cleanName + ",grocery/all?lock=" + lockId;

        ImageRequest request = new ImageRequest.Builder(this)
            .data(imageUrl)
            .diskCachePolicy(CachePolicy.ENABLED)
            .build();
        Coil.imageLoader(this).enqueue(request);
    }
}

private void updateTotals() {
    double totalItems = 0;
    double totalPrice = 0;
    for (ShoppingItem item : shoppingList) {
        totalItems += item.getQuantity();
        totalPrice += (item.getQuantity() * item.getPrice());
    }
    textViewTotalItems.setText(String.format(Locale.getDefault(), "Total de Itens: %.2f", totalItems));
    textViewTotalPrice.setText(NumberFormat.getCurrencyInstance(ptBr).format(totalPrice));
}
```

Método `preloadImages()`

`private void preloadImages()`

👉 Pré-carrega as imagens dos produtos para melhorar desempenho e reduzir tempo de carregamento.

Verifica se a lista existe

`if (shoppingList == null) return;`

👉 Se não houver produtos cadastrados, o método encerra.

Percorre todos os produtos

`for (ShoppingItem item : shoppingList)`

👉 Percorre cada objeto `ShoppingItem` da lista.

Trata o nome do produto

```
String cleanName = item.getName()
    .toLowerCase()
    .trim()
    .replace(" ", ",");
```

👉 Prepara o nome para gerar a URL.

Exemplo:

"Coca Cola"

vira:

"coca,cola"

Gera um identificador único

```
int lockId = Math.abs(
    item.getName().toLowerCase().trim().hashCode() % 1000);
```

👉 Cria um número baseado no nome do produto.

Objetivo:

- manter a mesma imagem para o mesmo produto
-

Monta a URL da imagem

```
String imageUrl = "https://loremflickr.com/300/300/"
    + cleanName
    + ",grocery/all?lock="
    + lockId;
```

👉 Gera automaticamente uma imagem relacionada ao produto.

Exemplo:

<https://loremflickr.com/300/300/arroz,grocery/all>

Cria requisição usando Coil

```
ImageRequest request =
    new ImageRequest.Builder(this)
```

👉 Cria uma solicitação de carregamento.

Ativa cache

```
.diskCachePolicy(CachePolicy.ENABLED)
```

👉 Salva imagens localmente.

Benefícios:

- menos consumo de internet
 - carregamento mais rápido
-

Inicia carregamento

```
Coil.imageLoader(this).enqueue(request);
```

👉 Coloca a imagem na fila para ser carregada.

Método `updateTotals()`

```
private void updateTotals()
```

👉 Atualiza automaticamente a quantidade total e o valor total da compra.

Variáveis acumuladoras

```
double totalItems = 0;
```

```
double totalPrice = 0;
```

👉 Iniciam os cálculos.

Percorre todos os produtos

```
for (ShoppingItem item : shoppingList)
```

👉 Analisa cada item cadastrado.

Soma quantidades

```
totalItems += item.getQuantity();
```

👉 Soma todos os produtos.

Soma preço total

```
totalPrice +=  
(item.getQuantity() * item.getPrice());
```

👉 Multiplica:

Quantidade × Preço

Exemplo:

2 × R\$10 = R\$20

Atualiza textos da tela

```
textViewTotalItems.setText(...)
```

👉 Mostra quantidade total.

```
textViewTotalPrice.setText(...)
```

👉 Mostra o valor total em formato brasileiro.

Exemplo:

R\$ 150,50

```

private void showItemDialog(ShoppingItem item, int position) {
    AlertDialog.Builder builder = new AlertDialog.Builder(this);
    LayoutInflater inflater = getLayoutInflater();
    View dialogView = inflater.inflate(R.layout.dialog_shopping_item, null);
    builder.setView(dialogView);

    EditText editTextName = dialogView.findViewById(R.id.editTextName);
    EditText editTextQuantity = dialogView.findViewById(R.id.editTextQuantity);
    Spinner spinnerUnit = dialogView.findViewById(R.id.spinnerUnit);
    EditText editTextPrice = dialogView.findViewById(R.id.editTextPrice);
    EditText editTextLocation = dialogView.findViewById(R.id.editTextLocation);

    String[] units = {"pacotes", "kilos", "gramas", "unidades", "latas", "garrafas", "barras", "outros"};
    ArrayAdapter<String> unitAdapter = new ArrayAdapter<>(this, android.R.layout.simple_spinner_item, units);
    unitAdapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
    spinnerUnit.setAdapter(unitAdapter);

    addCapitalizerWatcher(editTextName);
    addCapitalizerWatcher(editTextLocation);

    editTextPrice.addTextChangedListener(new TextWatcher() {
        private String current = "";
        @Override
        public void beforeTextChanged(CharSequence s, int start, int count, int after) {}
        @Override
        public void onTextChanged(CharSequence s, int start, int before, int count) {
            if (!s.toString().equals(current)) {
                editTextPrice.removeTextChangedListener(this);
                String cleanString = s.toString().replaceAll("[^\\d]", "");
                if (cleanString.isEmpty()) cleanString = "0";
                double parsed = Double.parseDouble(cleanString);
                String formatted = NumberFormat.getCurrencyInstance(ptBr).format((parsed / 100));
                current = formatted;
                editTextPrice.setText(formatted);
                editTextPrice.setSelection(formatted.length());
                editTextPrice.addTextChangedListener(this);
            }
        }
        @Override
        public void afterTextChanged(Editable s) {}
    });
}

```

Cria a caixa de diálogo

AlertDialog.Builder builder = new AlertDialog.Builder(this);

👉 Cria uma janela (AlertDialog) para interação do usuário.

Carrega o layout do formulário

View dialogView = inflater.inflate(
R.layout.dialog_shopping_item, null);

👉 Carrega o XML:

dialog_shopping_item.xml

Esse layout contém:

- Nome
 - Quantidade
 - Unidade
 - Preço
 - Local
-

Liga XML ao Java

Exemplo:

```
EditText editTextName =  
dialogView.findViewById(R.id.editTextName);
```

👉 Faz o Java acessar os componentes do formulário.

Cria lista de unidades

```
String[] units = {  
"pacotes",  
"kilos",  
"gramas",  
"unidades",  
"latas",  
"garrafas",  
"barras",  
"outros"  
};
```

👉 Define as opções disponíveis para o usuário.

Configura o Spinner

```
ArrayAdapter<String> unitAdapter =  
new ArrayAdapter<>(this,  
android.R.layout.simple_spinner_item,  
units);
```

👉 Preenche o **Spinner** com as unidades.

Exemplo:

- pacotes

- kilos
 - unidades
-

Capitaliza textos automaticamente

```
addCapitalizeWatcher(editTextName);
```

```
addCapitalizeWatcher(editTextLocation);
```

👉 Faz a primeira letra ficar maiúscula automaticamente.

Exemplo:

arroz

vira:

Arroz

Monitora mudanças no preço

```
editTextPrice.addTextChangedListener(  
    new TextWatcher()
```

👉 Observa alterações no campo de preço.

Remove caracteres inválidos

```
String cleanString =  
s.toString().replaceAll("[^\\d]", "");
```

👉 Remove tudo que não for número.

Exemplo:

R\$10,50

vira:

1050

Formata para moeda

```
String formatted =
```



```
NumberFormat.getCurrencyInstance(ptBr)  
.format(parsed / 100);
```

👉 Converte automaticamente para padrão brasileiro.

Exemplo:

1050

vira:

R\$ 10,50

Atualiza o campo

```
editTextPrice.setText(formatted);
```

👉 Mostra o valor formatado na tela.

Slide 12 - MainActivity

```
if (item != null) {
    builder.setTitle("Editar Produto");
    editTextName.setText(item.getName());
    editTextQuantity.setText(String.valueOf(item.getQuantity()));
    long priceInCents = Math.round(item.getPrice() * 100);
    editTextPrice.setText(String.valueOf(priceInCents));
    editTextLocation.setText(item.getLocation());
    for (int i = 0; i < units.length; i++) {
        if (units[i].equals(item.getUnit())) {
            spinnerUnit.setSelection(i);
            break;
        }
    }
} else {
    builder.setTitle("Novo Produto");
    editTextPrice.setText("0");
}

builder.setPositiveButton("Salvar", (dialog, which) -> {
    String name = editTextName.getText().toString().trim();
    String loc = editTextLocation.getText().toString().trim();
    String qtyStr = editTextQuantity.getText().toString().replace(",", ".").trim();
    String priceStr = editTextPrice.getText().toString()
        .replaceAll("[R$\\s]", "").replace(".", "").replace(",", ".");

    if (!name.isEmpty() && !qtyStr.isEmpty()) {
        try {
            double qty = Double.parseDouble(qtyStr);
            double price = Double.parseDouble(priceStr);
            String unit = spinnerUnit.getSelectedItem().toString();

            if (item == null) {
                shoppingList.add(new ShoppingItem(name, qty, unit, price, loc));
                adapter.notifyItemInserted(shoppingList.size() - 1);
            } else {
                item.setName(name);
                item.setQuantity(qty);
                item.setUnit(unit);
                item.setPrice(price);
                item.setLocation(loc);
                adapter.notifyItemChanged(position);
            }
            saveData();
            updateTotals();
            preloadImages();
        } catch (NumberFormatException e) {
            Toast.makeText(this, "Erro nos valores", Toast.LENGTH_SHORT).show();
        }
    }
});
builder.setNegativeButton("Cancelar", null);
builder.create().show();
}
```

Verifica se é edição ou cadastro

if (item != null)

👉 Verifica se um produto foi enviado ao método.

- `item != null` → editar produto
 - `item == null` → criar novo produto
-

Caso seja edição

```
builder.setTitle("Editar Produto");
```

👉 Altera o título da janela.

Preenche os campos automaticamente

```
editTextName.setText(item.getName());
```

```
editTextQuantity.setText(  
String.valueOf(item.getQuantity()));
```

```
editTextLocation.setText(  
item.getLocation());
```

👉 Coloca os dados atuais do produto nos campos.

Exemplo:

Antes:

Nome: Arroz

Quantidade: 2

Local: Mercado

O usuário verá essas informações já preenchidas.

Seleciona a unidade correta

```
for (int i = 0; i < units.length; i++)
```

👉 Percorre todas as unidades disponíveis.

```
if (units[i].equals(item.getUnit()))
```

👉 Compara a unidade do produto com a lista.

```
spinnerUnit.setSelection(i);
```

👉 Seleciona automaticamente a opção correta.

Caso seja novo produto

```
else {  
    builder.setTitle("Novo Produto");  
    editTextPrice.setText("0");  
}
```

👉 Define o título como **Novo Produto** e inicia o preço com zero.

Botão Salvar

```
builder.setPositiveButton("Salvar", ...)
```

👉 Executa o salvamento.

Captura os valores digitados

```
String name =  
editTextName.getText().toString().trim();
```

👉 Obtém os dados digitados pelo usuário.

Trata o valor monetário

```
String priceStr =  
editTextPrice.getText()  
.toString()  
.replaceAll("[R$\\s]", "")  
.replace(".", "")  
.replace(",", ".");
```

👉 Remove:

- R\$
- espaços
- símbolos

Para permitir conversão em número.

Converte textos para números

```
double qty =  
Double.parseDouble(qtyStr);
```

```
double price =  
Double.parseDouble(priceStr);
```

👉 Transforma texto em valores numéricos.

Se for novo produto

```
shoppingList.add(  
new ShoppingItem(  
name,  
qty,  
unit,  
price,  
loc));
```

👉 Cria um novo objeto **ShoppingItem** e adiciona à lista.

Se for edição

```
item.setName(name);
```

```
item.setQuantity(qty);
```

```
item.setUnit(unit);
```

```
item.setPrice(price);
```

```
item.setLocation(loc);
```

👉 Atualiza os dados do produto existente.

Atualiza o RecyclerView

```
adapter.notifyItemInserted(...)
```

ou

```
adapter.notifyItemChanged(...)
```

👉 Atualiza a tela automaticamente.

Salva e atualiza o sistema

`saveData();`

`updateTotals();`

`preloadImages();`

👉 Executa:

- salvar dados
 - recalcular totais
 - carregar imagens
-

Tratamento de erro

`catch(NumberFormatException e)`

👉 Evita falhas caso valores inválidos sejam digitados.

`Toast.makeText(...)`

👉 Mostra uma mensagem:

Erro nos valores

Botão Cancelar

`builder.setNegativeButton("Cancelar", null);`

👉 Fecha a janela sem salvar.

```

private void addCapitalizeWatcher(EditText editText) {
    editText.addTextChangedListener(new TextWatcher() {
        private boolean isUpdating = false;
        @Override
        public void beforeTextChanged(CharSequence s, int start, int count, int after) {}
        @Override
        public void onTextChanged(CharSequence s, int start, int before, int count) {
            if (isUpdating) return;
            String original = s.toString();
            String capitalized = capitalizeWords(original);
            if (!original.equals(capitalized)) {
                isUpdating = true;
                int selection = editText.getSelectionStart();
                editText.setText(capitalizeWords(original));
                if (selection <= capitalized.length()) editText.setSelection(selection);
                isUpdating = false;
            }
        }
        @Override
        public void afterTextChanged(Editable s) {}
    });
}

private String capitalizeWords(String input) {
    if (input == null || input.isEmpty()) return input;
    StringBuilder capitalized = new StringBuilder();
    boolean nextTitleCase = true;
    for (char c : input.toCharArray()) {
        if (Character.isSpaceChar(c)) { nextTitleCase = true; }
        else if (nextTitleCase) { c = Character.toUpperCase(c); nextTitleCase = false; }
        capitalized.append(c);
    }
    return capitalized.toString();
}

@Override
public void onItemClick(int position) { showItemDialog(shoppingList.get(position), position); }

```

Método `addCapitalizeWatcher()`

`private void addCapitalizeWatcher(EditText editText)`

👉 Adiciona um monitor (`TextWatcher`) ao campo de texto.

Objetivo:

- transformar automaticamente a primeira letra de cada palavra em maiúscula.

Exemplo:

arroz integral

vira:

Arroz Integral

Adiciona observador do texto

```
editText.addTextChangedListener(new TextWatcher())
```

👉 Monitora tudo que o usuário digita.

Evita loop infinito

```
private boolean isUpdating = false;
```

👉 Sem isso, ao alterar o texto automaticamente, o `TextWatcher` chamaria ele mesmo várias vezes.

Detecta mudança no texto

```
String original = s.toString();
```

```
String capitalized = capitalizeWords(original);
```

👉 Pega o texto digitado e chama o método que converte letras.

Atualiza o campo

```
editText.setText(capitalized);
```

👉 Substitui o texto digitado pela versão formatada.

Método `capitalizeWords()`

```
private String capitalizeWords(String input)
```

👉 Recebe um texto e transforma a primeira letra de cada palavra em maiúscula.

Percorre cada caractere


```
for (char c : input.toCharArray())
```

👉 Analisa letra por letra.

Verifica espaços

```
if (Character.isSpaceChar(c))
```

👉 Quando encontra um espaço:

- entende que a próxima letra inicia uma nova palavra.
-

Converte para maiúsculo

```
c = Character.toUpperCase(c);
```

👉 Transforma a primeira letra em maiúscula.

Método `onItemClick()`

```
@Override
```

```
public void onItemClick(int position)
```

👉 Implementa a interface criada no `ShoppingAdapter`.

Quando o usuário clicar em um item:

```
showItemDialog(  
shoppingList.get(position),  
position);
```

👉 Abre a janela de edição do produto selecionado.

Fluxo:

Usuário clica no card

↓

Adapter detecta clique

↓

`onItemClick()` é chamado

↓

`showItemDialog()` abre a edição

Slide 13 - MainActivity

```
@Override
public void onDeleteClick(int position) {
    new AlertDialog.Builder(this)
        .setTitle("Remover Item")
        .setMessage("Deseja remover " + shoppingList.get(position).getName() + "?")
        .setPositiveButton("Sim", (dialog, which) -> {
            shoppingList.remove(position);
            adapter.notifyItemRemoved(position);
            adapter.notifyItemRangeChanged(position, shoppingList.size());
            saveData();
            updateTotals();
        }).setNegativeButton("Não", null).show();
}

private void saveData() {
    SharedPreferences sharedPreferences = getSharedPreferences(PREFS_NAME, MODE_PRIVATE);
    SharedPreferences.Editor editor = sharedPreferences.edit();
    Gson gson = new Gson();
    editor.putString(LIST_KEY, gson.toJson(shoppingList));
    editor.apply();
}

private void loadData() {
    SharedPreferences sharedPreferences = getSharedPreferences(PREFS_NAME, MODE_PRIVATE);
    String json = sharedPreferences.getString(LIST_KEY, null);
    Type type = new TypeToken<ArrayList<ShoppingItem>>().getType();
    shoppingList = new Gson().fromJson(json, type);
    if (shoppingList == null) {
        shoppingList = new ArrayList<>();
        shoppingList.add(new ShoppingItem("Coca Cola", 1.0, "unidades", 8.50, "Bebidas"));
        shoppingList.add(new ShoppingItem("Snickers", 1.0, "unidades", 5.50, "Doces"));
    }
}
```

Método `onDeleteClick()`

`@Override`

`public void onDeleteClick(int position)`

👉 É chamado quando o usuário clica no botão excluir.

Esse método implementa a interface criada no `ShoppingAdapter`.

Cria caixa de confirmação

`new AlertDialog.Builder(this)`

👉 Abre uma janela para confirmar a exclusão.

Define o título

```
.setTitle("Remover Item")
```

👉 Exibe:

Remover Item

Mostra mensagem

```
.setMessage(  
"Deseja remover " +  
shoppingList.get(position).getName()  
+ "?"  
)
```

👉 Pega o nome do produto na posição clicada.

Exemplo:

Deseja remover Coca Cola?

Botão "Sim"

```
.setPositiveButton("Sim", ...)
```

👉 Se o usuário confirmar:

Remove da lista

```
shoppingList.remove(position);
```

👉 Remove o objeto **ShoppingItem**.

Atualiza RecyclerView

```
adapter.notifyItemRemoved(position);
```

👉 Remove o card visual da tela.

```
adapter.notifyItemRangeChanged(  
position,  
shoppingList.size()  
);
```

👉 Reorganiza as posições da lista.

Salva alterações

```
saveData();
```

👉 Atualiza o armazenamento local.

Atualiza totais

```
updateTotals();
```

👉 Recalcula:

- total de itens
 - valor total
-

Botão "Não"

```
.setNegativeButton("Não", null)
```

👉 Fecha a janela sem excluir.

Método `saveData()`

```
private void saveData()
```

👉 Salva a lista localmente no celular.

Obtém SharedPreferences

```
SharedPreferences sharedPreferences =  
getSharedPreferences(  
    PREFS_NAME,  
    MODE_PRIVATE  
);
```

👉 Acessa o armazenamento interno.

Cria editor

```
SharedPreferences.Editor editor =  
sharedPreferences.edit();
```

👉 Permite gravar dados.

Converte lista para JSON

```
Gson gson = new Gson();
```

```
editor.putString(  
LIST_KEY,  
gson.toJson(shoppingList)  
);
```

👉 Transforma:

```
List<ShoppingItem>
```

em:

```
[  
{  
  "name": "Coca Cola",  
  "quantity": 1  
}  
]
```

Salva

```
editor.apply();
```

👉 Grava os dados no celular.

Método `loadData()`

```
private void loadData()
```

👉 Carrega os dados salvos ao abrir o aplicativo.

Recupera JSON salvo

```
String json =  
sharedPreferences.getString(  
LIST_KEY,
```

```
null  
);
```

👉 Lê os dados armazenados.

Define tipo da lista

```
Type type =  
new TypeToken<  
ArrayList<ShoppingItem>>() {}  
.getType();
```

👉 Informa ao Gson que será uma lista de **ShoppingItem**.

Converte JSON em objetos

```
shoppingList =  
new Gson().fromJson(  
json,  
type  
);
```

👉 Converte novamente para objetos Java.

Verifica se existem dados

```
if (shoppingList == null)
```

👉 Se não houver dados salvos:

cria uma lista vazia.

Adiciona produtos padrão

```
shoppingList.add(  
new ShoppingItem(  
"Coca Cola",  
1.0,  
"unidades",  
8.50,  
"Bebidas"  
));  
shoppingList.add(  

```

```
new ShoppingItem(  
    "Snickers",  
    1.0,  
    "unidades",  
    5.50,  
    "Doces"  
));
```

👉 Produtos iniciais exibidos na primeira execução.

Fluxo completo do sistema

Abrir aplicativo

↓

`loadData()` carrega dados

↓

Usuário adiciona/edita/remove produtos

↓

`saveData()` salva alterações

↓

RecyclerView atualiza tela

↓

`updateTotals()` recalcula valores

Slide 14 - build.gradle.kts

```
plugins {
    alias(libs.plugins.android.application)
}

android {
    namespace = "com.example.appgerencia"
    compileSdk = 35

    defaultConfig {
        applicationId = "com.example.appgerencia"
        minSdk = 28
        targetSdk = 35
        versionCode = 1
        versionName = "1.0"

        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        release {
            isMinifyEnabled = false
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }
}

dependencies {
    implementation(libs.appcompat)
    implementation(libs.material)
    implementation(libs.activity)
    implementation(libs.constraintlayout)
    implementation(libs.gson)
    implementation(libs.coil)
    implementation(libs.coil.svg)
    testImplementation(libs.junit)
    androidTestImplementation(libs.ext.junit)
    androidTestImplementation(libs.espresso.core)
}
```


Plugins

```
plugins {  
  
    alias(libs.plugins.android.application)  
  
}
```

👉 Ativa funcionalidades necessárias para criar um aplicativo Android.

Sem isso, o Android Studio não conseguiria compilar o projeto.

Configuração Android

```
android {
```

👉 Define configurações gerais do aplicativo.

Namespace

```
namespace = "com.example.appgerencia"
```

👉 Define o pacote principal do projeto.

É usado para identificar a aplicação.

SDK de compilação

```
compileSdk = 35
```

👉 Define a versão do Android usada para compilar o aplicativo.

Nesse caso:

- Android API 35
-

Configurações padrão

```
defaultConfig {
```

👉 Define informações básicas do aplicativo.

ID do aplicativo

`applicationId = "com.example.appgerencia"`

👉 Identificador único do app.

É semelhante a um CPF do aplicativo.

Versão mínima

`minSdk = 28`

👉 Define a menor versão Android suportada.

Nesse caso:

- Android 9 (API 28)
-

Versão alvo

`targetSdk = 35`

👉 Informa para qual versão Android o aplicativo foi otimizado.

Controle de versão

`versionCode = 1`

`versionName = "1.0"`

👉 Controlam versões do aplicativo.

- `versionCode` → número interno
 - `versionName` → versão exibida ao usuário
-

Build Types

`buildTypes {`

👉 Configura tipos de compilação.

Release

```
release {
```

👉 Configura a versão final do aplicativo.

Minificação

```
isMinifyEnabled = false
```

👉 Define se o código será reduzido e otimizado.

false:

- não remove código
 - facilita depuração
-

ProGuard

```
proguardFiles(  
    getDefaultProguardFile(  
        "proguard-android-optimize.txt"),  
        "proguard-rules.pro"  
    )
```

👉 Define regras de proteção e otimização do código.

Compatibilidade Java

```
compileOptions {
```

👉 Configura a versão Java usada.

```
sourceCompatibility = JavaVersion.VERSION_11
```

targetCompatibility = JavaVersion.VERSION_11

👉 O projeto usa Java 11.

Dependências

dependencies {

👉 Bibliotecas externas utilizadas no projeto.

AppCompat

implementation(libs.appcompat)

👉 Suporte para telas Android modernas.

Usado em:

extends AppCompatActivity

Material Design

implementation(libs.material)

👉 Componentes visuais do Google.

Usado em:

- FloatingActionButton
 - AlertDialog
 - temas
-

Activity

implementation(libs.activity)

👉 Controle de Activities.

ConstraintLayout

implementation(libs.constraintlayout)

👉 Organização de layouts complexos.

Gson

implementation(libs.gson)

👉 Converte objetos Java em JSON.

No sistema:

gson.toJson(shoppingList)

e

new Gson().fromJson(...)

Coil

implementation(libs.coil)

implementation(libs.coil.svg)

👉 Biblioteca de carregamento de imagens.

No projeto:

- baixa imagens automaticamente
 - usa cache
 - melhora desempenho
-

Bibliotecas de teste

testImplementation(libs.junit)

androidTestImplementation(libs.ext.junit)

androidTestImplementation(libs.espresso.core)

👉 Utilizadas para testes automáticos.

- JUnit → testes unitários
- Espresso → testes de interface